

Experiment ‘p_13_10_recursive_area_polygon_array’ Results

June 5, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 4

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 4

Problem Specification

Problem name: p_13_10_recursive_area_polygon_array

Natural language statement: null

Method signature: p_13_10_recursive_area_polygon_array(points: array<(real, real)>) returns (area: real)

Ensures

- `area == polygon_area(points[..])`

Requires

- `points.Length >= 3`

Functional Code Given

```
function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
```

```
var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);  
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;  
triangle_area + polygon_area([points[0]] + points[2..])  
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_10_recursive_area_polygon_array(points: array<(real, real)>) returns (area: real)`

The method should respect the following contract:

`requires points.Length >= 3, ensures area == polygon_area(points[..])`

The contract uses the following dafny code:

```
function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;
    triangle_area + polygon_area([points[0]] + points[2..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPOn9XNMnbxXKTmA3ocQ1CJONqxR

Model: gpt-5.5-2026-04-23

Created at: 1780668077

Finish reason: stop

Usage: 896 tokens in, and 6441 tokens out

```
{
  var n := points.Length;
  area := 0.0;
  var i := 1;

  assert 3 <= n;
  assert points[..] == points[..1] + points[1..];
  assert points[..1] == [points[0]];
  assert [points[0]] + points[1..] == points[..];

  while i < n - 2
    invariant n == points.Length
    invariant 3 <= n
    invariant 1 <= i <= n - 2
    invariant |[points[0]] + points[i..]| >= 3
    invariant area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..])
  {
    var (x1, y1) := points[0];
    var (x2, y2) := points[i];
    var (x3, y3) := points[i + 1];
    var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

    ghost var rem := [points[0]] + points[i..];
    assert i + 1 < n;
    assert |rem| == 1 + n - i;
    assert |rem| > 3;
    assert points[i..] == points[i..i + 1] + points[i + 1..];
    assert points[i..i + 1] == [points[i]];
    assert rem == [points[0]] + [points[i]] + points[i + 1..];
    assert rem[0] == points[0];
    assert rem[1] == points[i];
    assert rem[2] == points[i + 1];
    assert rem[2..] == points[i + 1..];
    assert [rem[0]] + rem[2..] == [points[0]] + points[i + 1..];
    assert polygon_area(rem) == triangle_area + polygon_area([points[0]] + points[i +
      ↪ 1..]);
    assert area + triangle_area + polygon_area([points[0]] + points[i + 1..]) ==
      ↪ polygon_area(points[..]);

    area := area + triangle_area;
    i := i + 1;
  }

  assert i == n - 2;
  assert i + 1 < n;

  var (x1, y1) := points[0];
```

```

var (x2, y2) := points[i];
var (x3, y3) := points[i + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

ghost var rem := [points[0]] + points[i..];
assert |rem| == 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];
assert polygon_area(rem) == triangle_area;
assert area + triangle_area == polygon_area(points[..]);
area := area + triangle_area;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(59,29): Error: assertion
  ↪ might not hold
|
59 | assert polygon_area(rem) == triangle_area + polygon_area([points[0]] + points[i +
  ↪ 1..]);
| ^^

```

Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPRLMhRZGDETyErsbDJBRhGH9v4H

Model: gpt-5.5-2026-04-23

Created at: 1780668235

Finish reason: stop

Usage: 1581 tokens in, and 3706 tokens out

```

{
  var n := points.Length;
  area := 0.0;
  var i := 1;

  assert 3 <= n;
  assert points[..] == points[..1] + points[1..];
  assert points[..1] == [points[0]];
  assert [points[0]] + points[1..] == points[..];
}

```

```

while i < n - 2
  invariant n == points.Length
  invariant 3 <= n
  invariant 1 <= i <= n - 2
  invariant |[points[0]] + points[i..]| >= 3
  invariant area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..])
{
  var (x1, y1) := points[0];
  var (x2, y2) := points[i];
  var (x3, y3) := points[i + 1];
  var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
  var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

  ghost var rem := [points[0]] + points[i..];
  assert i + 1 < n;
  assert |rem| == 1 + n - i;
  assert |rem| > 3;
  assert points[i..] == points[i..i + 1] + points[i + 1..];
  assert points[i..i + 1] == [points[i]];
  assert rem == [points[0]] + [points[i]] + points[i + 1..];
  assert rem[0] == points[0];
  assert rem[1] == points[i];
  assert rem[2] == points[i + 1];
  assert rem[0] == (x1, y1);
  assert rem[1] == (x2, y2);
  assert rem[2] == (x3, y3);
  assert rem[2..] == points[i + 1..];
  assert [rem[0]] + rem[2..] == [points[0]] + points[i + 1..];

  if sign < 0.0 {
    assert triangle_area == -sign / 2.0;
    assert polygon_area(rem) == -sign / 2.0 + polygon_area([rem[0]] + rem[2..]);
  } else {
    assert triangle_area == sign / 2.0;
    assert polygon_area(rem) == sign / 2.0 + polygon_area([rem[0]] + rem[2..]);
  }
  assert polygon_area(rem) == triangle_area + polygon_area([rem[0]] + rem[2..]);
  assert polygon_area(rem) == triangle_area + polygon_area([points[0]] + points[i +
    ↪ 1..]);
  assert area + triangle_area + polygon_area([points[0]] + points[i + 1..]) ==
    ↪ polygon_area(points[..]);

  area := area + triangle_area;
  i := i + 1;
}

assert i == n - 2;
assert i + 1 < n;

var (x1, y1) := points[0];

```

```

var (x2, y2) := points[i];
var (x3, y3) := points[i + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

ghost var rem := [points[0]] + points[i..];
assert |rem| == 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];
assert rem[0] == (x1, y1);
assert rem[1] == (x2, y2);
assert rem[2] == (x3, y3);
if sign < 0.0 {
  assert triangle_area == -sign / 2.0;
  assert polygon_area(rem) == -sign / 2.0;
} else {
  assert triangle_area == sign / 2.0;
  assert polygon_area(rem) == sign / 2.0;
}
assert polygon_area(rem) == triangle_area;
assert area + triangle_area == polygon_area(points[..]);
area := area + triangle_area;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(51,4): Error: assertion
  ↪ might not hold
|

```

```

51 | assert points[i..] == points[i..i + 1] + points[i + 1..];
| ~~~~~

```

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(65,31): Error: assertion
  ↪ might not hold
|

```

```

65 | assert polygon_area(rem) == -sign / 2.0 + polygon_area([rem[0]] + rem[2..]);
| ^^

```

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(68,31): Error: assertion
  ↪ might not hold
|

```

```

68 | assert polygon_area(rem) == sign / 2.0 + polygon_area([rem[0]] + rem[2..]);
| ^^

```

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(100,29): Error: assertion
  ↪ might not hold
|

```

```

100 | assert polygon_area(rem) == sign / 2.0;

```

| ^^

Dafny program verifier finished with 2 verified, 4 errors

Correct the error in code by altering only the method body. Produce and show only the

- ↪ Dafny body, including the curly braces that surround it. Do not show the signature
- ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPSxdJOXVbVKTe2zYansjNeKMu1g

Model: gpt-5.5-2026-04-23

Created at: 1780668335

Finish reason: stop

Usage: 3190 tokens in, and 5983 tokens out

```
{
  var n := points.Length;
  area := 0.0;
  var i := 1;

  assert points[..] == [points[0]] + points[1..];

  while i < n - 2
    invariant n == points.Length
    invariant 3 <= n
    invariant 1 <= i <= n - 2
    invariant |[points[0]] + points[i..]| >= 3
    invariant area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..])
    decreases n - i
  {
    assert 0 <= i < n;
    assert 0 <= i + 1 <= n;

    ghost var tail := points[i..];
    assert |tail| == n - i;
    assert 1 <= |tail|;
    assert tail == tail[..1] + tail[1..];
    assert |tail[..1]| == 1;
    assert tail[..1][0] == points[i];
    assert tail[..1] == [points[i]];
    assert |tail[1..]| == |points[i + 1..]|;
    forall k | 0 <= k < |tail[1..]|
      ensures tail[1..][k] == points[i + 1..][k]
    {
      assert tail[1..][k] == tail[1 + k];
      assert tail[1 + k] == points[i + 1 + k];
      assert points[i + 1..][k] == points[i + 1 + k];
    }
  }
}
```



```

assert tail[1..] == points[i + 1..];
assert points[i..] == [points[i]] + points[i + 1..];

var (x1, y1) := points[0];
var (x2, y2) := points[i];
var (x3, y3) := points[i + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

ghost var rem := [points[0]] + points[i..];
assert rem == [points[0]] + ([points[i]] + points[i + 1..]);
assert |rem| == 1 + n - i;
assert |rem| > 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];

assert |rem[2..]| == |points[i + 1..]|;
forall k | 0 <= k < |rem[2..]|
  ensures rem[2..][k] == points[i + 1..][k]
{
  assert rem[2..][k] == rem[2 + k];
  assert rem[2 + k] == ([points[i]] + points[i + 1..])[1 + k];
  assert ([points[i]] + points[i + 1..])[1 + k] == points[i + 1..][k];
}
assert rem[2..] == points[i + 1..];
assert [rem[0]] + rem[2..] == [points[0]] + points[i + 1..];
assert |[rem[0]] + rem[2..]| >= 3;

ghost var (rx1, ry1) := rem[0];
ghost var (rx2, ry2) := rem[1];
ghost var (rx3, ry3) := rem[2];
ghost var rsign: real := rx1 * (ry2 - ry3) + rx2 * (ry3 - ry1) + rx3 * (ry1 - ry2);
ghost var rtri := if rsign < 0.0 then -rsign / 2.0 else rsign / 2.0;
assert rsign == sign;
assert rtri == triangle_area;
assert |rem| != 3;
assert polygon_area(rem) == rtri + polygon_area([rem[0]] + rem[2..]);
assert polygon_area(rem) == triangle_area + polygon_area([points[0]] + points[i +
  ↪ 1..]);
assert area + triangle_area + polygon_area([points[0]] + points[i + 1..]) ==
  ↪ polygon_area(points[..]);

area := area + triangle_area;
i := i + 1;
assert area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..]);
}

assert i == n - 2;
assert 0 <= i < n;
assert 0 <= i + 1 < n;

```

```

ghost var tail := points[i..];
assert |tail| == 2;
assert tail[0] == points[i];
assert tail[1] == points[i + 1];

var (x1, y1) := points[0];
var (x2, y2) := points[i];
var (x3, y3) := points[i + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

ghost var rem := [points[0]] + points[i..];
assert |rem| == 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];

ghost var (rx1, ry1) := rem[0];
ghost var (rx2, ry2) := rem[1];
ghost var (rx3, ry3) := rem[2];
ghost var rsign: real := rx1 * (ry2 - ry3) + rx2 * (ry3 - ry1) + rx3 * (ry1 - ry2);
ghost var rtri := if rsign < 0.0 then -rsign / 2.0 else rsign / 2.0;
assert rsign == sign;
assert rtri == triangle_area;
assert polygon_area(rem) == rtri;
assert polygon_area(rem) == triangle_area;
assert area + triangle_area == polygon_area(points[..]);
area := area + triangle_area;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(94,29): Error: assertion
  ↪ might not hold
|
94 | assert polygon_area(rem) == rtri + polygon_area([rem[0]] + rem[2..]);
|   ^^

```

Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPV7uPMP3rT94P7hstFeTeRRQbOQ

Model: gpt-5.5-2026-04-23

Created at: 1780668469

Finish reason: stop

Usage: 4746 tokens in, and 6100 tokens out

```
{
  var n := points.Length;
  area := 0.0;
  var i := 1;

  assert points[..] == [points[0]] + points[1..];

  while i < n - 2
    invariant n == points.Length
    invariant 3 <= n
    invariant 1 <= i <= n - 2
    invariant |[points[0]] + points[i..]| >= 3
    invariant area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..])
    decreases n - i
  {
    assert 0 <= i < n;
    assert 0 <= i + 1 <= n;

    ghost var tail := points[i..];
    assert |tail| == n - i;
    assert 1 <= |tail|;
    assert tail == tail[..1] + tail[1..];
    assert |tail[..1]| == 1;
    assert tail[..1][0] == points[i];
    assert tail[..1] == [points[i]];
    assert |tail[1..]| == |points[i + 1..]|;
    forall k | 0 <= k < |tail[1..]|
      ensures tail[1..][k] == points[i + 1..][k]
    {
      assert tail[1..][k] == tail[1 + k];
      assert tail[1 + k] == points[i + 1 + k];
      assert points[i + 1..][k] == points[i + 1 + k];
    }
    assert tail[1..] == points[i + 1..];
    assert points[i..] == [points[i]] + points[i + 1..];

    var (x1, y1) := points[0];
    var (x2, y2) := points[i];
    var (x3, y3) := points[i + 1];
    var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

    ghost var rem := [points[0]] + points[i..];
```

```

assert rem == [points[0]] + ([points[i]] + points[i + 1..]);
assert |rem| == 1 + n - i;
assert |rem| > 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];

assert |rem[2..]| == |points[i + 1..]|;
forall k | 0 <= k < |rem[2..]|
  ensures rem[2..][k] == points[i + 1..][k]
{
  assert rem[2..][k] == rem[2 + k];
  assert rem[2 + k] == ([points[i]] + points[i + 1..])[1 + k];
  assert ([points[i]] + points[i + 1..])[1 + k] == points[i + 1..][k];
}
assert rem[2..] == points[i + 1..];
assert [rem[0]] + rem[2..] == [points[0]] + points[i + 1..];
assert |[rem[0]] + rem[2..]| >= 3;

ghost var (rx1, ry1) := rem[0];
ghost var (rx2, ry2) := rem[1];
ghost var (rx3, ry3) := rem[2];
ghost var rsign: real := rx1 * (ry2 - ry3) + rx2 * (ry3 - ry1) + rx3 * (ry1 - ry2);
ghost var rtri := if rsign < 0.0 then -rsign / 2.0 else rsign / 2.0;
assert rem[0] == (rx1, ry1);
assert rem[1] == (rx2, ry2);
assert rem[2] == (rx3, ry3);
assert rsign == sign;
assert rtri == triangle_area;
assert |rem| != 3;
assert polygon_area(rem) == (if |rem| == 3 then rtri else rtri + polygon_area([rem[0]]
  ↪ + rem[2..]));
assert polygon_area(rem) == rtri + polygon_area([rem[0]] + rem[2..]);
assert polygon_area(rem) == triangle_area + polygon_area([points[0]] + points[i +
  ↪ 1..]);
assert area + triangle_area + polygon_area([points[0]] + points[i + 1..]) ==
  ↪ polygon_area(points[..]);

area := area + triangle_area;
i := i + 1;
assert area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..]);
}

assert i == n - 2;
assert 0 <= i < n;
assert 0 <= i + 1 < n;

ghost var tail := points[i..];
assert |tail| == 2;
assert tail[0] == points[i];
assert tail[1] == points[i + 1];

```

```

var (x1, y1) := points[0];
var (x2, y2) := points[i];
var (x3, y3) := points[i + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

ghost var rem := [points[0]] + points[i..];
assert |rem| == 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];

ghost var (rx1, ry1) := rem[0];
ghost var (rx2, ry2) := rem[1];
ghost var (rx3, ry3) := rem[2];
ghost var rsign: real := rx1 * (ry2 - ry3) + rx2 * (ry3 - ry1) + rx3 * (ry1 - ry2);
ghost var rtri := if rsign < 0.0 then -rsign / 2.0 else rsign / 2.0;
assert rem[0] == (rx1, ry1);
assert rem[1] == (rx2, ry2);
assert rem[2] == (rx3, ry3);
assert rsign == sign;
assert rtri == triangle_area;
assert polygon_area(rem) == (if |rem| == 3 then rtri else rtri + polygon_area([rem[0]]
    ↪ + rem[2..]));
assert polygon_area(rem) == rtri;
assert polygon_area(rem) == triangle_area;
assert area + triangle_area == polygon_area(points[..]);
area := area + triangle_area;
}

```

Final Program

```

function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;

```

```

    triangle_area + polygon_area([points[0]] + points[2..])
}

method p_13_10_recursive_area_polygon_array(points: array<(real, real)>) returns (area:
    ↪ real)
    requires points.Length >= 3
    ensures area == polygon_area(points[..])
{
    var n := points.Length;
    area := 0.0;
    var i := 1;

    assert points[..] == [points[0]] + points[1..];

    while i < n - 2
        invariant n == points.Length
        invariant 3 <= n
        invariant 1 <= i <= n - 2
        invariant |[points[0]] + points[i..]| >= 3
        invariant area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..])
        decreases n - i
    {
        assert 0 <= i < n;
        assert 0 <= i + 1 <= n;

        ghost var tail := points[i..];
        assert |tail| == n - i;
        assert 1 <= |tail|;
        assert tail == tail[..1] + tail[1..];
        assert |tail[..1]| == 1;
        assert tail[..1][0] == points[i];
        assert tail[..1] == [points[i]];
        assert |tail[1..]| == |points[i + 1..]|;
        forall k | 0 <= k < |tail[1..]|
            ensures tail[1..][k] == points[i + 1..][k]
        {
            assert tail[1..][k] == tail[1 + k];
            assert tail[1 + k] == points[i + 1 + k];
            assert points[i + 1..][k] == points[i + 1 + k];
        }
        assert tail[1..] == points[i + 1..];
        assert points[i..] == [points[i]] + points[i + 1..];

        var (x1, y1) := points[0];
        var (x2, y2) := points[i];
        var (x3, y3) := points[i + 1];
        var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
        var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

        ghost var rem := [points[0]] + points[i..];
        assert rem == [points[0]] + ([points[i]] + points[i + 1..]);
    }
}

```

```

assert |rem| == 1 + n - i;
assert |rem| > 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];

assert |rem[2..]| == |points[i + 1..]|;
forall k | 0 <= k < |rem[2..]|
  ensures rem[2..][k] == points[i + 1..][k]
{
  assert rem[2..][k] == rem[2 + k];
  assert rem[2 + k] == ([points[i]] + points[i + 1..])[1 + k];
  assert ([points[i]] + points[i + 1..])[1 + k] == points[i + 1..][k];
}
assert rem[2..] == points[i + 1..];
assert [rem[0]] + rem[2..] == [points[0]] + points[i + 1..];
assert |[rem[0]] + rem[2..]| >= 3;

ghost var (rx1, ry1) := rem[0];
ghost var (rx2, ry2) := rem[1];
ghost var (rx3, ry3) := rem[2];
ghost var rsign: real := rx1 * (ry2 - ry3) + rx2 * (ry3 - ry1) + rx3 * (ry1 - ry2);
ghost var rtri := if rsign < 0.0 then -rsign / 2.0 else rsign / 2.0;
assert rem[0] == (rx1, ry1);
assert rem[1] == (rx2, ry2);
assert rem[2] == (rx3, ry3);
assert rsign == sign;
assert rtri == triangle_area;
assert |rem| != 3;
assert polygon_area(rem) == (if |rem| == 3 then rtri else rtri + polygon_area([rem[0]]
  ↪ + rem[2..]));
assert polygon_area(rem) == rtri + polygon_area([rem[0]] + rem[2..]);
assert polygon_area(rem) == triangle_area + polygon_area([points[0]] + points[i +
  ↪ 1..]);
assert area + triangle_area + polygon_area([points[0]] + points[i + 1..]) ==
  ↪ polygon_area(points[..]);

area := area + triangle_area;
i := i + 1;
assert area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..]);
}

assert i == n - 2;
assert 0 <= i < n;
assert 0 <= i + 1 < n;

ghost var tail := points[i..];
assert |tail| == 2;
assert tail[0] == points[i];
assert tail[1] == points[i + 1];

```

```

var (x1, y1) := points[0];
var (x2, y2) := points[i];
var (x3, y3) := points[i + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

ghost var rem := [points[0]] + points[i..];
assert |rem| == 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];

ghost var (rx1, ry1) := rem[0];
ghost var (rx2, ry2) := rem[1];
ghost var (rx3, ry3) := rem[2];
ghost var rsign: real := rx1 * (ry2 - ry3) + rx2 * (ry3 - ry1) + rx3 * (ry1 - ry2);
ghost var rtri := if rsign < 0.0 then -rsign / 2.0 else rsign / 2.0;
assert rem[0] == (rx1, ry1);
assert rem[1] == (rx2, ry2);
assert rem[2] == (rx3, ry3);
assert rsign == sign;
assert rtri == triangle_area;
assert polygon_area(rem) == (if |rem| == 3 then rtri else rtri + polygon_area([rem[0]]
    ↪ + rem[2..]));
assert polygon_area(rem) == rtri;
assert polygon_area(rem) == triangle_area;
assert area + triangle_area == polygon_area(points[..]);
area := area + triangle_area;
}

```

Total Token Usage

Input tokens: 10413

Output tokens: 22230

Reasoning tokens: 16761

Sum of 'total tokens': 32643

Experiment Timings

Overall Experiment started at 1780668076607, ended at 1780668643967, lasting 567360ms (567.36 seconds)

Iteration #4 started at 1780668468671, ended at 1780668643966, lasting 175295ms (175.30 seconds)

Iteration #1 started at 1780668076610, ended at 1780668234586, lasting 157976ms (157.98 seconds)

Iteration #2 started at 1780668234669, ended at 1780668335070, lasting 100401ms (100.40 seconds)

Iteration #3 started at 1780668335071, ended at 1780668468671, lasting 133600ms (133.60 seconds)

Experiment ‘p_13_10_recursive_area_polygon_seq’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 4

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 4

Problem Specification

Problem name: p_13_10_recursive_area_polygon_seq

Natural language statement: null

Method signature: p_13_10_recursive_area_polygon_seq(points: seq<(real, real)>) returns (area: real)

Ensures

- `area == polygon_area(points)`

Requires

- `|points| >= 3`

Functional Code Given

```
function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
```

```
var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);  
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;  
triangle_area + polygon_area([points[0]] + points[2..])  
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_10_recursive_area_polygon_seq(points: seq<(real, real)>) returns (area: real)`

The method should respect the following contract:

`requires |points| >= 3, ensures area == polygon_area(points)`

The contract uses the following dafny code:

```
function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;
    triangle_area + polygon_area([points[0]] + points[2..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPHTPShQ4RYY6aUWsm0rG57Xlv5C

Model: gpt-5.5-2026-04-23

Created at: 1780667623

Finish reason: stop

Usage: 896 tokens in, and 1367 tokens out

```
{
  if |points| == 3 {
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;
  } else {
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

    var remaining := [points[0]] + points[2..];
    assert |remaining| == |points| - 1;
    assert |remaining| >= 3;

    var rest := p_13_10_recursive_area_polygon_seq(remaining);
    area := triangle_area + rest;
  }
}
```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

problem_data/p_13_10_recursive_area_polygon_seq/program.dfy(24,0): Error: a postcondition
↪ could not be proved on this return path

```
|
24 | {
| ^
```

problem_data/p_13_10_recursive_area_polygon_seq/program.dfy(23,14): Related location:
↪ this is the postcondition that could not be proved

```
|
23 | ensures area == polygon_area(points)
| ^^
```

problem_data/p_13_10_recursive_area_polygon_seq/program.dfy(42,50): Error: cannot prove
↪ termination; try supplying a decreases clause

```
|
42 | var rest := p_13_10_recursive_area_polygon_seq(remaining);
| ^
```

Dafny program verifier finished with 2 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
→ Dafny body, including the curly braces that surround it. Do not show the signature
→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPILT6Cf7mNf4JCK1Geqb0j6Jgs7

Model: gpt-5.5-2026-04-23

Created at: 1780667677

Finish reason: stop

Usage: 1270 tokens in, and 3541 tokens out

```
{
  var acc: real := 0.0;
  var q := points;

  while |q| > 3
    invariant |q| >= 3
    invariant acc + polygon_area(q) == polygon_area(points)
    decreases |q|
  {
    var (x1, y1) := q[0];
    var (x2, y2) := q[1];
    var (x3, y3) := q[2];
    var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

    var qNext := [q[0]] + q[2..];
    assert |qNext| == |q| - 1;
    assert |qNext| >= 3;
    assert polygon_area(q) == triangle_area + polygon_area(qNext);

    acc := acc + triangle_area;
    q := qNext;
  }

  assert |q| == 3;
  var (x1, y1) := q[0];
  var (x2, y2) := q[1];
  var (x3, y3) := q[2];
  var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
  var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;
  assert polygon_area(q) == triangle_area;

  area := acc + triangle_area;
  assert area == polygon_area(points);
}
```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```
problem_data/p_13_10_recursive_area_polygon_seq/program.dfy(42,27): Error: assertion
  ↳ might not hold
  |
42 | assert polygon_area(q) == triangle_area + polygon_area(qNext);
  | ^^
```

Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
↳ Dafny body, including the curly braces that surround it. Do not show the signature
↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPK5KTNdS5kUiYwmGFwhz00VAAFE

Model: gpt-5.5-2026-04-23

Created at: 1780667785

Finish reason: stop

Usage: 1745 tokens in, and 4051 tokens out

```
{
  var acc: real := 0.0;
  var q := points;
  assert q == points;
  assert acc + polygon_area(q) == polygon_area(points);

  while |q| > 3
    invariant |q| >= 3
    invariant acc + polygon_area(q) == polygon_area(points)
    decreases |q|
  {
    var (x1, y1) := q[0];
    var (x2, y2) := q[1];
    var (x3, y3) := q[2];
    var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

    var qNext := [q[0]] + q[2..];
    assert 4 <= |q|;
    assert |q[2..]| == |q| - 2;
    assert |qNext| == |q| - 1;
    assert |qNext| >= 3;
    assert qNext == [q[0]] + q[2..];
    assert q[0] == (x1, y1);
    assert q[1] == (x2, y2);
    assert q[2] == (x3, y3);
```

```

    calc {
      polygon_area(q);
    == {
      assert |q| != 3;
      assert q[0] == (x1, y1);
      assert q[1] == (x2, y2);
      assert q[2] == (x3, y3);
    }
      triangle_area + polygon_area([q[0]] + q[2..]);
    == {
      assert qNext == [q[0]] + q[2..];
    }
      triangle_area + polygon_area(qNext);
    }

    calc {
      (acc + triangle_area) + polygon_area(qNext);
    == {
    }
      acc + (triangle_area + polygon_area(qNext));
    == {
      assert polygon_area(q) == triangle_area + polygon_area(qNext);
    }
      acc + polygon_area(q);
    == {
      assert acc + polygon_area(q) == polygon_area(points);
    }
      polygon_area(points);
    }

    acc := acc + triangle_area;
    q := qNext;
  }

  assert |q| == 3;
  var (x1, y1) := q[0];
  var (x2, y2) := q[1];
  var (x3, y3) := q[2];
  var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
  var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;
  assert q[0] == (x1, y1);
  assert q[1] == (x2, y2);
  assert q[2] == (x3, y3);

  calc {
    polygon_area(q);
  == {
    assert |q| == 3;
    assert q[0] == (x1, y1);
    assert q[1] == (x2, y2);

```



```

    assert q[2] == (x3, y3);
  }
  triangle_area;
}

area := acc + triangle_area;
calc {
  area;
== {
}
  acc + triangle_area;
== {
  assert polygon_area(q) == triangle_area;
}
  acc + polygon_area(q);
== {
  assert acc + polygon_area(q) == polygon_area(points);
}
  polygon_area(points);
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_13_10_recursive_area_polygon_seq/program.dfy(59,20): Error: the
  ↪ calculation step between the previous line and this line could not be proved
|
59 | triangle_area + polygon_area([q[0]] + q[2..]);
|   ^

```

Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPLnw7HDd7LUiurYOJapoCNm187L

Model: gpt-5.5-2026-04-23

Created at: 1780667891

Finish reason: stop

Usage: 2853 tokens in, and 7905 tokens out

```

{
  var acc: real := 0.0;
  var q := points;

```

```

assert acc + polygon_area(q) == polygon_area(points);

while |q| > 3
  invariant |q| >= 3
  invariant acc + polygon_area(q) == polygon_area(points)
  decreases |q|
{
  var sign: real := (q[0]).0 * ((q[1]).1 - (q[2]).1) + (q[1]).0 * ((q[2]).1 - (q[0]).1)
    ↪ + (q[2]).0 * ((q[0]).1 - (q[1]).1);
  var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

  var qNext := [q[0]] + q[2..];
  assert |q[2..]| == |q| - 2;
  assert |qNext| == |q| - 1;
  assert |qNext| >= 3;
  assert qNext == [q[0]] + q[2..];

  assert polygon_area(q) == triangle_area + polygon_area(qNext) by {
    reveal polygon_area();
    assert |q| != 3;
    assert qNext == [q[0]] + q[2..];
  }

  var newAcc := acc + triangle_area;
  assert newAcc + polygon_area(qNext) == acc + (triangle_area + polygon_area(qNext));
  assert newAcc + polygon_area(qNext) == acc + polygon_area(q);
  assert newAcc + polygon_area(qNext) == polygon_area(points);

  acc := newAcc;
  q := qNext;
}

assert |q| == 3;
var sign: real := (q[0]).0 * ((q[1]).1 - (q[2]).1) + (q[1]).0 * ((q[2]).1 - (q[0]).1) +
  ↪ (q[2]).0 * ((q[0]).1 - (q[1]).1);
var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

assert polygon_area(q) == triangle_area by {
  reveal polygon_area();
  assert |q| == 3;
}

area := acc + triangle_area;
assert area == acc + polygon_area(q);
assert area == polygon_area(points);
}

```

Final Program

```

function polygon_area(points: seq<(real, real)>): real

```

```

decreases |points|
requires |points| >= 3
ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;
    triangle_area + polygon_area([points[0]] + points[2..])
  }
}

method p_13_10_recursive_area_polygon_seq(points: seq<(real, real)>) returns (area: real)
  requires |points| >= 3
  ensures area == polygon_area(points)
{
  var acc: real := 0.0;
  var q := points;
  assert acc + polygon_area(q) == polygon_area(points);

  while |q| > 3
    invariant |q| >= 3
    invariant acc + polygon_area(q) == polygon_area(points)
    decreases |q|
    {
      var sign: real := (q[0]).0 * ((q[1]).1 - (q[2]).1) + (q[1]).0 * ((q[2]).1 - (q[0]).1)
        ↪ + (q[2]).0 * ((q[0]).1 - (q[1]).1);
      var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

      var qNext := [q[0]] + q[2..];
      assert |q[2..]| == |q| - 2;
      assert |qNext| == |q| - 1;
      assert |qNext| >= 3;
      assert qNext == [q[0]] + q[2..];

      assert polygon_area(q) == triangle_area + polygon_area(qNext) by {
        reveal polygon_area();
        assert |q| != 3;
        assert qNext == [q[0]] + q[2..];
      }

      var newAcc := acc + triangle_area;
      assert newAcc + polygon_area(qNext) == acc + (triangle_area + polygon_area(qNext));
      assert newAcc + polygon_area(qNext) == acc + polygon_area(q);
    }
}

```

```

    assert newAcc + polygon_area(qNext) == polygon_area(points);

    acc := newAcc;
    q := qNext;
}

assert |q| == 3;
var sign: real := (q[0]).0 * ((q[1]).1 - (q[2]).1) + (q[1]).0 * ((q[2]).1 - (q[0]).1) +
    ↪ (q[2]).0 * ((q[0]).1 - (q[1]).1);
var triangle_area: real := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

assert polygon_area(q) == triangle_area by {
    reveal polygon_area();
    assert |q| == 3;
}

area := acc + triangle_area;
assert area == acc + polygon_area(q);
assert area == polygon_area(points);
}

```

Total Token Usage

Input tokens: 6764

Output tokens: 16864

Reasoning tokens: 14386

Sum of 'total tokens': 23628

Experiment Timings

Overall Experiment started at 1780667623432, ended at 1780668070384, lasting 446952ms (446.95 seconds)

Iteration #4 started at 1780667890319, ended at 1780668070384, lasting 180065ms (180.07 seconds)

Iteration #1 started at 1780667623433, ended at 1780667676577, lasting 53144ms (53.14 seconds)

Iteration #2 started at 1780667676580, ended at 1780667784433, lasting 107853ms (107.85 seconds)

Iteration #3 started at 1780667784433, ended at 1780667890298, lasting 105865ms (105.87 seconds)

Experiment ‘p_13_1_recursive_rectangle_area’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_1_recursive_rectangle_area

Natural language statement: null

Method signature: p_13_1_recursive_rectangle_area(width: nat, height: nat) returns (area: nat)

Ensures

- `area == width * height`

Requires

- `1 <= width`
- `1 <= height`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_1_recursive_rectangle_area(width: nat, height: nat) returns (area: nat)`

The method should respect the following contract:

`requires 1 <= width, requires 1 <= height, ensures area == width * height`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOmn7tLHaGgPo438kWbFLHTJipQo

Model: gpt-5.5-2026-04-23

Created at: 1780665721

Finish reason: stop

Usage: 276 tokens in, and 490 tokens out

```
{
  if height == 1 {
    area := width;
  } else {
    var smallerArea := p_13_1_recursive_rectangle_area(width, height - 1);
    area := smallerArea + width;
  }
}
```

Final Program

```
method p_13_1_recursive_rectangle_area(width: nat, height: nat) returns (area: nat)
  requires 1 <= width
  requires 1 <= height
  ensures area == width * height
{
  if height == 1 {
    area := width;
  } else {
```

```
    var smallerArea := p_13_1_recursive_rectangle_area(width, height - 1);  
    area := smallerArea + width;  
  }  
}
```

Total Token Usage

Input tokens: 276

Output tokens: 490

Reasoning tokens: 418

Sum of 'total tokens': 766

Experiment Timings

Overall Experiment started at 1780665701034, ended at 1780665776386, lasting 75352ms (75.35 seconds)

Iteration #1 started at 1780665701634, ended at 1780665776386, lasting 74752ms (74.75 seconds)

Experiment 'p_13_2_recursive_square_area' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_2_recursive_square_area

Natural language statement: null

Method signature: p_13_2_recursive_square_area(width: nat) returns (area: nat)

Ensures

- `area == width * width`

Requires

- `1 <= width`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_2_recursive_square_area(width: nat) returns (area: nat)`

The method should respect the following contract:

`requires 1 <= width, ensures area == width * width`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOo4CEkZ4rGrPtousPTi4Nq7Wvrr

Model: gpt-5.5-2026-04-23

Created at: 1780665800

Finish reason: stop

Usage: 256 tokens in, and 1510 tokens out

```
{  
  area := width * width;  
}
```

Final Program

```
method p_13_2_recursive_square_area(width: nat) returns (area: nat)  
  requires 1 <= width  
  ensures area == width * width  
{  
  area := width * width;  
}
```

Total Token Usage

Input tokens: 256

Output tokens: 1510

Reasoning tokens: 1470

Sum of ‘total tokens’: 1766

Experiment Timings

Overall Experiment started at 1780665797441, ended at 1780665892979, lasting 95538ms (95.54 seconds)

Iteration #1 started at 1780665797444, ended at 1780665892972, lasting 95528ms (95.53 seconds)

Experiment 'p_13_3_recursive_reverse_string' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_3_recursive_reverse_string

Natural language statement: null

Method signature: p_13_3_recursive_reverse_string(text: string) returns (reversed: string)

Ensures

- `|reversed| == |text|`
- `forall i :: 0 <= i < |text| ==> reversed[i] == text[|text| - 1 - i]`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_3_recursive_reverse_string(text: string) returns (reversed: string)`

The method should respect the following contract:

`ensures |reversed| == |text|, ensures forall i :: 0 <= i < |text| ==> reversed[i] == text`
↪ `[|text| - 1 - i]`

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOpengDIDFEQzELPZFlxrNjgHHVn

Model: gpt-5.5-2026-04-23

Created at: 1780665898

Finish reason: stop

Usage: 310 tokens in, and 5206 tokens out

```
{
  reversed := "";
  var i := 0;
  while i < |text|
    invariant 0 <= i <= |text|
    invariant |reversed| == i
    invariant forall j :: 0 <= j < i ==> reversed[j] == text[i - 1 - j]
  {
    assert 0 <= i < |text|;
    var prev := reversed;
    reversed := text[i..i+1] + prev;
    assert |text[i..i+1]| == 1;
    assert (text[i..i+1])[0] == text[i];
    assert |reversed| == i + 1;

    forall j | 0 <= j < i + 1
      ensures reversed[j] == text[i - j]
    {
      if j == 0 {
        assert reversed[j] == (text[i..i+1])[0];
      }
    }
  }
}
```

```

    assert (text[i..i+1])[0] == text[i];
    assert i - j == i;
    assert reversed[j] == text[i - j];
  } else {
    assert 0 <= j - 1 < i;
    assert reversed[j] == prev[j - 1];
    assert prev[j - 1] == text[i - 1 - (j - 1)];
    assert i - 1 - (j - 1) == i - j;
    assert reversed[j] == text[i - j];
  }
}

i := i + 1;
}
}

```

Final Program

```

method p_13_3_recursive_reverse_string(text: string) returns (reversed: string)
  ensures |reversed| == |text|
  ensures forall i :: 0 <= i < |text| ==> reversed[i] == text[|text| - 1 - i]
{
  reversed := "";
  var i := 0;
  while i < |text|
    invariant 0 <= i <= |text|
    invariant |reversed| == i
    invariant forall j :: 0 <= j < i ==> reversed[j] == text[i - 1 - j]
  {
    assert 0 <= i < |text|;
    var prev := reversed;
    reversed := text[i..i+1] + prev;
    assert |text[i..i+1]| == 1;
    assert (text[i..i+1])[0] == text[i];
    assert |reversed| == i + 1;

    forall j | 0 <= j < i + 1
      ensures reversed[j] == text[i - j]
    {
      if j == 0 {
        assert reversed[j] == (text[i..i+1])[0];
        assert (text[i..i+1])[0] == text[i];
        assert i - j == i;
        assert reversed[j] == text[i - j];
      } else {
        assert 0 <= j - 1 < i;
        assert reversed[j] == prev[j - 1];
        assert prev[j - 1] == text[i - 1 - (j - 1)];
        assert i - 1 - (j - 1) == i - j;
        assert reversed[j] == text[i - j];
      }
    }
  }
}

```



```
    }  
  }  
  
  i := i + 1;  
}  
}
```

Total Token Usage

Input tokens: 310

Output tokens: 5206

Reasoning tokens: 4778

Sum of 'total tokens': 5516

Experiment Timings

Overall Experiment started at 1780665895067, ended at 1780666045546, lasting 150479ms (150.48 seconds)

Iteration #1 started at 1780665895068, ended at 1780666045546, lasting 150478ms (150.48 seconds)

Experiment 'p_13_6_recursive_find' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_6_recursive_find

Natural language statement: null

Method signature: p_13_6_recursive_find(text: string, str: string) returns (found: bool)

Ensures

- `found == find(text, str)`

Requires

- `1 <= |str| <= |text|`

Functional Code Given

```
function find(text: string, str: string): bool{
  if |text| < |str| then false
  else if text[0 .. |str|] == str then true
  else find(text[1..], str)
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_6_recursive_find(text: string, str: string) returns (found: bool)`

The method should respect the following contract:

`requires 1 <= |str| <= |text|, ensures found == find(text, str)`

The contract uses the following dafny code:

```
function find(text: string, str: string): bool{
  if |text| < |str| then false
  else if text[0 .. |str|] == str then true
  else find(text[1..], str)
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnSLsoCHhYtOi5ofkA4F7Fel1bFLW

Model: gpt-5.5-2026-04-23

Created at: 1780679428

Finish reason: stop

Usage: 426 tokens in, and 4828 tokens out

```
{
  if text[0 .. |str|] == str {
    found := true;
  } else if |text| == |str| {
    assert text[0 .. |str|] != str;
    assert |text[1..]| == |text| - 1;
    assert |text[1..]| < |str|;
    found := false;
  } else {
    assert |str| < |text|;
    assert |text[1..]| == |text| - 1;
```

```
    assert |str| <= |text[1..]|;  
    found := p_13_6_recursive_find(text[1..], str);  
  }  
}
```

Final Program

```
function find(text: string, str: string): bool{  
  if |text| < |str| then false  
  else if text[0 .. |str|] == str then true  
  else find(text[1..], str)  
}  
  
method p_13_6_recursive_find(text: string, str: string) returns (found: bool)  
  requires 1 <= |str| <= |text|  
  ensures found == find(text, str)  
{  
  if text[0 .. |str|] == str {  
    found := true;  
  } else if |text| == |str| {  
    assert text[0 .. |str|] != str;  
    assert |text[1..]| == |text| - 1;  
    assert |text[1..]| < |str|;  
    found := false;  
  } else {  
    assert |str| < |text|;  
    assert |text[1..]| == |text| - 1;  
    assert |str| <= |text[1..]|;  
    found := p_13_6_recursive_find(text[1..], str);  
  }  
}
```

Total Token Usage

Input tokens: 426

Output tokens: 4828

Reasoning tokens: 4608

Sum of 'total tokens': 5254

Experiment Timings

Overall Experiment started at 1780679421150, ended at 1780679580725, lasting 159575ms (159.58 seconds)

Iteration #1 started at 1780679421293, ended at 1780679580716, lasting 159423ms (159.42 seconds)

Experiment ‘p_13_7_recursive_index_of’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_7_recursive_index_of

Natural language statement: null

Method signature: p_13_7_recursive_index_of(text: string, str: string) returns (index: int)

Ensures

- `index == find_index(text, str, 0)`

Requires

- `1 <= |str| <= |text|`

Functional Code Given

```
function starts_with(text: string, str: string) : (b: bool)
  ensures b == (|str| <= |text| && forall i :: 0 <= i < |str| ==> text[i] == str[i])
{
  if |str| == 0 then true
  else if |text| == 0 then false
  else if text[0] == str[0] then starts_with(text[1..], str[1..])
  else false
}

function find_index(text: string, str: string, index: nat) : (ret : int)
  ensures starts_with(text, str) ==> ret == index
{
  if |text| < |str| then -1
  else if starts_with(text, str) then index
  else find_index(text[1..], str, index + 1)
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_7_recursive_index_of(text: string, str: string) returns (index: int)`

The method should respect the following contract:

`requires 1 <= |str| <= |text|, ensures index == find_index(text, str, 0)`

The contract uses the following dafny code:

```
function starts_with(text: string, str: string) : (b: bool)
  ensures b == (|str| <= |text| && forall i :: 0 <= i < |str| ==> text[i] == str[i])
{
  if |str| == 0 then true
  else if |text| == 0 then false
  else if text[0] == str[0] then starts_with(text[1..], str[1..])
  else false
}

function find_index(text: string, str: string, index: nat) : (ret : int)
  ensures starts_with(text, str) ==> ret == index
{
  if |text| < |str| then -1
  else if starts_with(text, str) then index
  else find_index(text[1..], str, index + 1)
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOzFuBfli8S4bCEZMo6Y3QoEiK9Q

Model: gpt-5.5-2026-04-23

Created at: 1780666493

Finish reason: stop

Usage: 694 tokens in, and 9054 tokens out


```

{
  var pos: nat := 0;
  assert text[0..] == text;

  while pos + |str| <= |text| && text[pos..pos + |str|] != str
    invariant pos <= |text|
    invariant 1 <= |str| <= |text|
    invariant find_index(text, str, 0) == find_index(text[pos..], str, pos)
    decreases |text| - pos
  {
    ghost var s := text[pos..];
    assert |str| <= |s|;
    assert s[..|str|] == text[pos..pos + |str|];
    assert s[..|str|] != str;

    if starts_with(s, str) {
      assert |str| <= |s| && forall k :: 0 <= k < |str| ==> s[k] == str[k];
      forall k | 0 <= k < |str|
        ensures s[..|str|][k] == str[k]
      {
        assert s[..|str|][k] == s[k];
      }
      assert |s[..|str|]| == |str|;
      assert s[..|str|] == str;
      assert false;
    }
    assert !starts_with(s, str);
    assert find_index(s, str, pos) == find_index(s[1..], str, pos + 1);
    assert s[1..] == text[pos + 1..];
    assert find_index(s, str, pos) == find_index(text[pos + 1..], str, pos + 1);
    assert find_index(text, str, 0) == find_index(text[pos + 1..], str, pos + 1);
    assert pos < |text|;
    pos := pos + 1;
  }

  assert !(pos + |str| <= |text| && text[pos..pos + |str|] != str);
  ghost var s := text[pos..];

  if pos + |str| <= |text| {
    assert text[pos..pos + |str|] == str;
    assert |str| <= |s|;
    assert s[..|str|] == text[pos..pos + |str|];
    assert s[..|str|] == str;
    forall k | 0 <= k < |str|
      ensures s[k] == str[k]
    {
      assert s[..|str|][k] == str[k];
      assert s[..|str|][k] == s[k];
    }
    assert forall k :: 0 <= k < |str| ==> s[k] == str[k];
    assert starts_with(s, str);
  }

```

```

    assert find_index(s, str, pos) == pos;
    index := pos;
  } else {
    assert pos + |str| > |text|;
    assert |s| == |text| - pos;
    assert |s| < |str|;
    assert find_index(s, str, pos) == -1;
    index := -1;
  }

  assert index == find_index(s, str, pos);
  assert find_index(text, str, 0) == find_index(s, str, pos);
  assert index == find_index(text, str, 0);
}

```

Final Program

```

function starts_with(text: string, str: string) : (b: bool)
  ensures b == (|str| <= |text| && forall i :: 0 <= i < |str| ==> text[i] == str[i])
{
  if |str| == 0 then true
  else if |text| == 0 then false
  else if text[0] == str[0] then starts_with(text[1..], str[1..])
  else false
}

function find_index(text: string, str: string, index: nat) : (ret : int)
  ensures starts_with(text, str) ==> ret == index
{
  if |text| < |str| then -1
  else if starts_with(text, str) then index
  else find_index(text[1..], str, index + 1)
}

method p_13_7_recursive_index_of(text: string, str: string) returns (index: int)
  requires 1 <= |str| <= |text|
  ensures index == find_index(text, str, 0)
{
  var pos: nat := 0;
  assert text[0..] == text;

  while pos + |str| <= |text| && text[pos..pos + |str|] != str
    invariant pos <= |text|
    invariant 1 <= |str| <= |text|
    invariant find_index(text, str, 0) == find_index(text[pos..], str, pos)
    decreases |text| - pos
  {
    ghost var s := text[pos..];
    assert |str| <= |s|;
    assert s[..|str|] == text[pos..pos + |str|];
  }
}

```

```

assert s[..|str|] != str;

if starts_with(s, str) {
  assert |str| <= |s| && forall k :: 0 <= k < |str| ==> s[k] == str[k];
  forall k | 0 <= k < |str|
    ensures s[..|str|][k] == str[k]
  {
    assert s[..|str|][k] == s[k];
  }
  assert |s[..|str|]| == |str|;
  assert s[..|str|] == str;
  assert false;
}
assert !starts_with(s, str);
assert find_index(s, str, pos) == find_index(s[1..], str, pos + 1);
assert s[1..] == text[pos + 1..];
assert find_index(s, str, pos) == find_index(text[pos + 1..], str, pos + 1);
assert find_index(text, str, 0) == find_index(text[pos + 1..], str, pos + 1);
assert pos < |text|;
pos := pos + 1;
}

assert !(pos + |str| <= |text| && text[pos..pos + |str|] != str);
ghost var s := text[pos..];

if pos + |str| <= |text| {
  assert text[pos..pos + |str|] == str;
  assert |str| <= |s|;
  assert s[..|str|] == text[pos..pos + |str|];
  assert s[..|str|] == str;
  forall k | 0 <= k < |str|
    ensures s[k] == str[k]
  {
    assert s[..|str|][k] == str[k];
    assert s[..|str|][k] == s[k];
  }
  assert forall k :: 0 <= k < |str| ==> s[k] == str[k];
  assert starts_with(s, str);
  assert find_index(s, str, pos) == pos;
  index := pos;
} else {
  assert pos + |str| > |text|;
  assert |s| == |text| - pos;
  assert |s| < |str|;
  assert find_index(s, str, pos) == -1;
  index := -1;
}

assert index == find_index(s, str, pos);
assert find_index(text, str, 0) == find_index(s, str, pos);
assert index == find_index(text, str, 0);

```

}

Total Token Usage

Input tokens: 694

Output tokens: 9054

Reasoning tokens: 8188

Sum of 'total tokens': 9748

Experiment Timings

Overall Experiment started at 1780666492215, ended at 1780666712744, lasting 220529ms (220.53 seconds)

Iteration #1 started at 1780666492249, ended at 1780666712744, lasting 220495ms (220.50 seconds)

Experiment ‘p_13_8_recursive_largest_array’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_8_recursive_largest_array

Natural language statement: null

Method signature: p_13_8_recursive_largest_array(inputs: array<int>) returns (largest: int)

Ensures

- largest == find_largest(inputs[..])

Requires

- inputs.Length >= 1

Functional Code Given

```
function find_largest(inputs: seq<int>): int
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> find_largest(inputs) >= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert inputs == [inputs[0]] + inputs[1..];
    var sub_largest := find_largest(inputs[1..]) ;
    if inputs[0] >= sub_largest then inputs[0] else sub_largest
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_8_recursive_largest_array(inputs: array<int>) returns (largest: int)`

The method should respect the following contract:

`requires inputs.Length >= 1, ensures largest == find_largest(inputs[..])`

The contract uses the following dafny code:

```
function find_largest(inputs: seq<int>): int
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> find_largest(inputs) >= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert inputs == [inputs[0]] + inputs[1..];
    var sub_largest := find_largest(inputs[1..]) ;
    if inputs[0] >= sub_largest then inputs[0] else sub_largest
  }
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnP3txdTl9DOLA9rECphojclkl17y

Model: gpt-5.5-2026-04-23

Created at: 1780666781

Finish reason: stop

Usage: 524 tokens in, and 6642 tokens out

```
{
  var i := inputs.Length - 1;
  largest := inputs[i];

  var init := inputs[i..];
  assert |init| == 1;
```

```

assert init[0] == inputs[i];
assert find_largest(init) == inputs[i];
assert largest == find_largest(inputs[i..]);

while i > 0
  invariant 0 <= i < inputs.Length
  invariant largest == find_largest(inputs[i..])
  decreases i
{
  var subLargest := largest;
  i := i - 1;

  assert 0 <= i < inputs.Length;
  assert i + 1 < inputs.Length;
  assert subLargest == find_largest(inputs[i + 1..]);

  var s := inputs[i..];
  assert |s| > 1;
  assert s == [s[0]] + s[1..];
  assert s[0] == inputs[i];
  assert s[1..] == inputs[i + 1..];
  assert find_largest(s) == (if s[0] >= find_largest(s[1..]) then s[0] else find_largest
    ↪ (s[1..]));
  assert find_largest(inputs[i..]) == (if inputs[i] >= subLargest then inputs[i] else
    ↪ subLargest);

  if inputs[i] >= subLargest {
    largest := inputs[i];
  } else {
    largest := subLargest;
  }

  assert largest == (if inputs[i] >= subLargest then inputs[i] else subLargest);
  assert largest == find_largest(inputs[i..]);
}

assert i == 0;
assert inputs[i..] == inputs[..];
assert largest == find_largest(inputs[..]);
}

```

Final Program

```

function find_largest(inputs: seq<int>): int
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> find_largest(inputs) >= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert inputs == [inputs[0]] + inputs[1..];

```



```

    var sub_largest := find_largest(inputs[1..]) ;
    if inputs[0] >= sub_largest then inputs[0] else sub_largest
}

method p_13_8_recursive_largest_array(inputs: array<int>) returns (largest: int)
    requires inputs.Length >= 1
    ensures largest == find_largest(inputs[..])
{
    var i := inputs.Length - 1;
    largest := inputs[i];

    var init := inputs[i..];
    assert |init| == 1;
    assert init[0] == inputs[i];
    assert find_largest(init) == inputs[i];
    assert largest == find_largest(inputs[i..]);

    while i > 0
        invariant 0 <= i < inputs.Length
        invariant largest == find_largest(inputs[i..])
        decreases i
    {
        var subLargest := largest;
        i := i - 1;

        assert 0 <= i < inputs.Length;
        assert i + 1 < inputs.Length;
        assert subLargest == find_largest(inputs[i + 1..]);

        var s := inputs[i..];
        assert |s| > 1;
        assert s == [s[0]] + s[1..];
        assert s[0] == inputs[i];
        assert s[1..] == inputs[i + 1..];
        assert find_largest(s) == (if s[0] >= find_largest(s[1..]) then s[0] else find_largest
            ↪ (s[1..]));
        assert find_largest(inputs[i..]) == (if inputs[i] >= subLargest then inputs[i] else
            ↪ subLargest);

        if inputs[i] >= subLargest {
            largest := inputs[i];
        } else {
            largest := subLargest;
        }

        assert largest == (if inputs[i] >= subLargest then inputs[i] else subLargest);
        assert largest == find_largest(inputs[i..]);
    }

    assert i == 0;
    assert inputs[i..] == inputs[..];

```

```
    assert largest == find_largest(inputs[...]);  
}
```

Total Token Usage

Input tokens: 524

Output tokens: 6642

Reasoning tokens: 6141

Sum of ‘total tokens’: 7166

Experiment Timings

Overall Experiment started at 1780666780499, ended at 1780666944671, lasting 164172ms (164.17 seconds)

Iteration #1 started at 1780666780500, ended at 1780666944521, lasting 164021ms (164.02 seconds)

Experiment 'p_13_8_recursive_largest_seq' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_8_recursive_largest_seq

Natural language statement: null

Method signature: p_13_8_recursive_largest_seq(inputs: seq<int>) returns (largest: int)

Ensures

- `largest == find_largest(inputs)`

Requires

- `|inputs| >= 1`

Functional Code Given

```
function find_largest(inputs: seq<int>): int
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> find_largest(inputs) >= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert inputs == [inputs[0]] + inputs[1..];
    var sub_largest := find_largest(inputs[1..]) ;
    if inputs[0] >= sub_largest then inputs[0] else sub_largest
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_8_recursive_largest_seq(inputs: seq<int>) returns (largest: int)`

The method should respect the following contract:

`requires |inputs| >= 1, ensures largest == find_largest(inputs)`

The contract uses the following dafny code:

```
function find_largest(inputs: seq<int>): int
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> find_largest(inputs) >= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert inputs == [inputs[0]] + inputs[1..];
    var sub_largest := find_largest(inputs[1..]) ;
    if inputs[0] >= sub_largest then inputs[0] else sub_largest
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnP2v1LZ449JjsLS9rWBUasbSLWVk

Model: gpt-5.5-2026-04-23

Created at: 1780666721

Finish reason: stop

Usage: 524 tokens in, and 648 tokens out

```
{
  if |inputs| == 1 {
    largest := inputs[0];
  } else {
    assert |inputs[1..]| >= 1;
    var sub_largest := p_13_8_recursive_largest_seq(inputs[1..]);
```

```

    assert inputs == [inputs[0]] + inputs[1..];
    if inputs[0] >= sub_largest {
        largest := inputs[0];
    } else {
        largest := sub_largest;
    }
}
}

```

Final Program

```

function find_largest(inputs: seq<int>): int
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> find_largest(inputs) >= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert inputs == [inputs[0]] + inputs[1..];
    var sub_largest := find_largest(inputs[1..]) ;
    if inputs[0] >= sub_largest then inputs[0] else sub_largest
  }

method p_13_8_recursive_largest_seq(inputs: seq<int>) returns (largest: int)
  requires |inputs| >= 1
  ensures largest == find_largest(inputs)
{
  if |inputs| == 1 {
    largest := inputs[0];
  } else {
    assert |inputs[1..]| >= 1;
    var sub_largest := p_13_8_recursive_largest_seq(inputs[1..]);
    assert inputs == [inputs[0]] + inputs[1..];
    if inputs[0] >= sub_largest {
      largest := inputs[0];
    } else {
      largest := sub_largest;
    }
  }
}
}

```

Total Token Usage

Input tokens: 524

Output tokens: 648

Reasoning tokens: 512

Sum of ‘total tokens’: 1172

Experiment Timings

Overall Experiment started at 1780666718757, ended at 1780666778446, lasting 59689ms (59.69 seconds)

Iteration #1 started at 1780666718887, ended at 1780666778446, lasting 59559ms (59.56 seconds)

Experiment 'p_13_9_recursive_sum_array' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_9_recursive_sum_array

Natural language statement: null

Method signature: p_13_9_recursive_sum_array(inputs: array<int>) returns (sum: int)

Ensures

- `sum == sum_array(inputs[..])`

Functional Code Given

```
function sum_array(inputs: seq<int>): int
  ensures sum_array(inputs) == (if |inputs| == 0 then 0 else inputs[0] + sum_array(inputs
    ↪ [1..]))
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_9_recursive_sum_array(inputs: array<int>) returns (sum: int)`

The method should respect the following contract:

`ensures sum == sum_array(inputs[..])`

The contract uses the following dafny code:

```
function sum_array(inputs: seq<int>): int
  ensures sum_array(inputs) == (if |inputs| == 0 then 0 else inputs[0] + sum_array(inputs
    ↪ [1..]))
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnSOOZt1MpGn3fZ8ftlv4ibjtYdmK

Model: gpt-5.5-2026-04-23

Created at: 1780679584

Finish reason: stop

Usage: 436 tokens in, and 3871 tokens out

```
{
  sum := 0;
  var i := inputs.Length;
  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant sum == sum_array(inputs[i..])
    decreases i
  {
    var oldI := i;
```

```

    i := i - 1;
    assert oldI == i + 1;
    assert sum == sum_array(inputs[oldI..]);
    assert sum == sum_array(inputs[i + 1..]);
    assert inputs[i..] == [inputs[i]] + inputs[i + 1..];
    assert inputs[i..][0] == inputs[i];
    assert inputs[i..][1..] == inputs[i + 1..];
    assert sum_array(inputs[i..]) == inputs[i] + sum_array(inputs[i + 1..]);
    sum := inputs[i] + sum;
    assert sum == sum_array(inputs[i..]);
}
assert i == 0;
assert inputs[i..] == inputs[..];
}

```

Final Program

```

function sum_array(inputs: seq<int>): int
  ensures sum_array(inputs) == (if |inputs| == 0 then 0 else inputs[0] + sum_array(inputs
    ↪ [1..]))
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}

method p_13_9_recursive_sum_array(inputs: array<int>) returns (sum: int)
  ensures sum == sum_array(inputs[..])
{
  sum := 0;
  var i := inputs.Length;
  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant sum == sum_array(inputs[i..])
    decreases i
  {
    var oldI := i;
    i := i - 1;
    assert oldI == i + 1;
    assert sum == sum_array(inputs[oldI..]);
    assert sum == sum_array(inputs[i + 1..]);
    assert inputs[i..] == [inputs[i]] + inputs[i + 1..];
    assert inputs[i..][0] == inputs[i];
    assert inputs[i..][1..] == inputs[i + 1..];
    assert sum_array(inputs[i..]) == inputs[i] + sum_array(inputs[i + 1..]);
    sum := inputs[i] + sum;
    assert sum == sum_array(inputs[i..]);
  }
  assert i == 0;
  assert inputs[i..] == inputs[..];
}

```

Total Token Usage

Input tokens: 436

Output tokens: 3871

Reasoning tokens: 3584

Sum of 'total tokens': 4307

Experiment Timings

Overall Experiment started at 1780679583763, ended at 1780679714155, lasting 130392ms (130.39 seconds)

Iteration #1 started at 1780679583764, ended at 1780679714155, lasting 130391ms (130.39 seconds)

Experiment 'p_13_9_recursive_sum_seq' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_9_recursive_sum_seq

Natural language statement: null

Method signature: p_13_9_recursive_sum_seq(inputs: seq<int>) returns (sum: int)

Ensures

- `sum == sum_array(inputs)`

Functional Code Given

```
function sum_array(inputs: seq<int>): int
  ensures sum_array(inputs) == (if |inputs| == 0 then 0 else inputs[0] + sum_array(inputs
    ↪ [1..]))
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_9_recursive_sum_seq(inputs: seq<int>) returns (sum: int)`

The method should respect the following contract:

`ensures sum == sum_array(inputs)`

The contract uses the following dafny code:

```
function sum_array(inputs: seq<int>): int
  ensures sum_array(inputs) == (if |inputs| == 0 then 0 else inputs[0] + sum_array(inputs
    ↪ [1..]))
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnP6d9whr2ur4dHaprxhkBuFwgFfA

Model: gpt-5.5-2026-04-23

Created at: 1780666951

Finish reason: stop

Usage: 434 tokens in, and 543 tokens out

```
{
  if |inputs| == 0 {
    sum := 0;
  } else {
    var tailSum := p_13_9_recursive_sum_seq(inputs[1..]);
    sum := inputs[0] + tailSum;
  }
}
```

Final Program

```
function sum_array(inputs: seq<int>): int
  ensures sum_array(inputs) == (if |inputs| == 0 then 0 else inputs[0] + sum_array(inputs
    ↪ [1..]))
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}

method p_13_9_recursive_sum_seq(inputs: seq<int>) returns (sum: int)
  ensures sum == sum_array(inputs)
{
  if |inputs| == 0 {
    sum := 0;
  } else {
    var tailSum := p_13_9_recursive_sum_seq(inputs[1..]);
    sum := inputs[0] + tailSum;
  }
}
```

Total Token Usage

Input tokens: 434

Output tokens: 543

Reasoning tokens: 466

Sum of 'total tokens': 977

Experiment Timings

Overall Experiment started at 1780666949798, ended at 1780667000909, lasting 51111ms (51.11 seconds)

Iteration #1 started at 1780666949919, ended at 1780667000908, lasting 50989ms (50.99 seconds)

